# EXPERIMENT 10
# MINI PROJECT REPORT
## End-to-End DevOps Pipeline for the QuickDrop / RentRide Web Application

*Subject: Agile Software Development and DevOps*

| Item | Details |
|---|---|
| Student | Yash Singh (Roll no. 2401118), Batch T22 |
| Project | QuickDrop REST API with RentRide vehicle-rental front end |
| GitHub repository | github.com/Yash-Singh18/ASD_MPR |
| DevOps tools | Jira, GitHub, GitHub Actions, Jenkins, Docker, Prometheus, Grafana, AWS |
| Date | 08 October 2026 |

## 1. Aim

To design and implement a complete Agile and DevOps workflow for a web application: plan the work in Jira, automate Continuous Integration with GitHub Actions, automate Continuous Delivery with Jenkins, package everything with Docker, monitor it with Prometheus and Grafana, and prepare the deployment on AWS.

## 2. Objectives

- Manage the project backlog and sprints using Agile (Scrum) practices in Jira.
- Build an automated CI pipeline that lints, tests and security-scans every change.
- Build a CD pipeline in Jenkins that builds Docker images and deploys the application.
- Containerise the front end, back end, database, Jenkins and monitoring stack.
- Observe the running system with Prometheus metrics and a Grafana dashboard.
- Prepare a repeatable deployment of the same stack to AWS.

## 3. Introduction

QuickDrop is a REST API built with FastAPI and PostgreSQL. It exposes endpoints for categories, products, a shopping cart and orders (including checkout and order status). The original grocery storefront was later re-skinned as RentRide, a vehicle rental site, and the seed data was changed from groceries to vehicles, without changing the API design.

The purpose of the project is not the application itself but the engineering process around it. Every change moves through the same path: a Jira ticket, a Git commit, automated checks on GitHub, an automated deployment from Jenkins, and live monitoring in Grafana. This shortens feedback time and keeps releases repeatable, which is the central idea of DevOps.

## 4. Tools and Technologies

| Tool | Purpose | Usage in this project |
|---|---|---|
| Jira | Agile planning, backlog, sprints | Tickets numbered QD-xx; backlog created from a CSV file |

| Tool | Purpose | Usage in this project |
|---|---|---|
| Git / GitHub | Version control and collaboration | Repository ASD_MPR with 35 commits and feature branches |
| GitHub Actions | Continuous Integration | Lint, CI, CodeQL, E2E smoke test, security scan, release workflows |
| Jenkins 2.580.1 | Continuous Delivery | Job quickdrop-deploy, run in Docker and configured with JCasC |
| Docker / Compose | Containerisation | Images for backend and frontend; one compose file for the full stack |
| Prometheus v2.55.1 | Metrics collection | Scrapes the /metrics endpoint of the FastAPI backend |
| Grafana 11.3.1 | Visualisation | Provisioned QuickDrop API dashboard |
| AWS | Cloud hosting | EC2 with Docker Compose, ECR and CloudWatch (ready to deploy) |

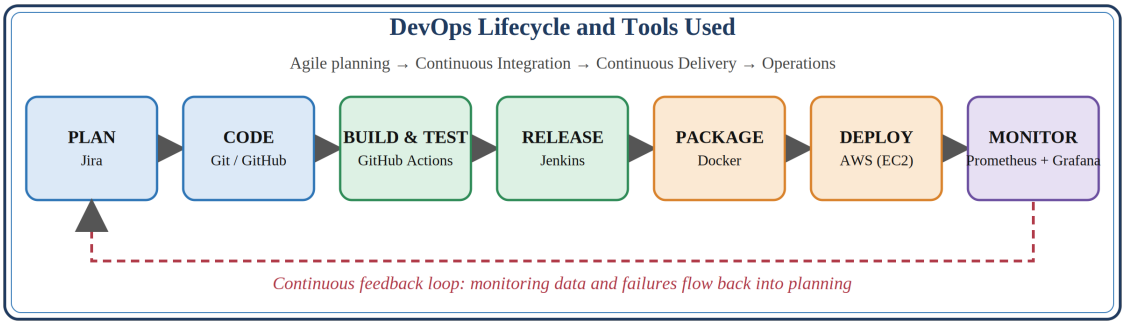



*Figure 1: DevOps lifecycle mapped to the tools used*

## 5. Agile Planning with Jira

The project followed Scrum. The product backlog was prepared first and imported into Jira from a CSV file (commit QD-7/QD-8). Work was then broken into small tickets with the key prefix QD, and every commit message starts with its ticket number so that code changes can be traced back to a requirement.

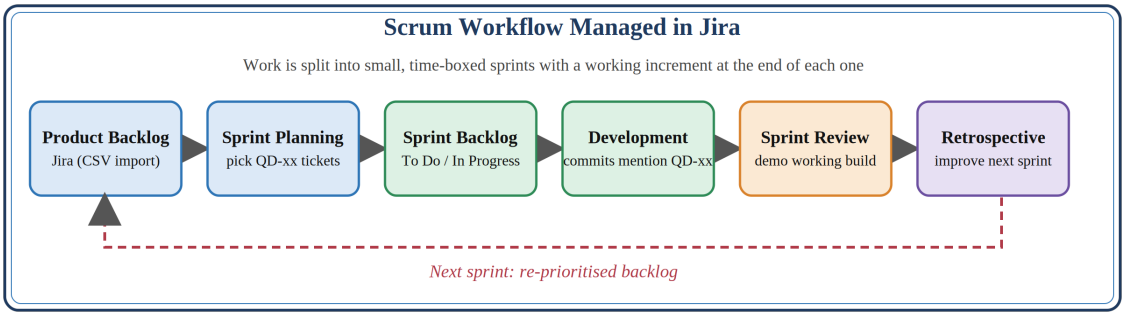



*Figure 2: Scrum workflow managed in Jira*

| Ticket | Work item (from commit history) | Area |
|---|---|---|
| QD-7, QD-8 | Repository skeleton, Jira backlog CSV and Scrum set-up | Planning |

| Ticket | Work item (from commit history) | Area |
|---|---|---|
| QD-27 | Run Jenkins in Docker with JCasC and create the deploy job | CD |
| QD-28 | Jenkinsfile fix: removed the timestamps option (plugin not installed) | CD |
| QD-31 | Provisioned Grafana dashboard and monitoring stack | Monitoring |
| QD-32 | README written; original project brief kept in docs/ | Documentation |

## 6. System Architecture

The application runs as a set of Docker containers on one Compose network. The browser talks to an nginx container serving the front end, which calls the FastAPI back end. The back end stores data in PostgreSQL and exposes /health and /metrics. Prometheus scrapes the metrics and Grafana reads them from Prometheus. Jenkins sits beside the stack and redeploys it with Docker Compose.

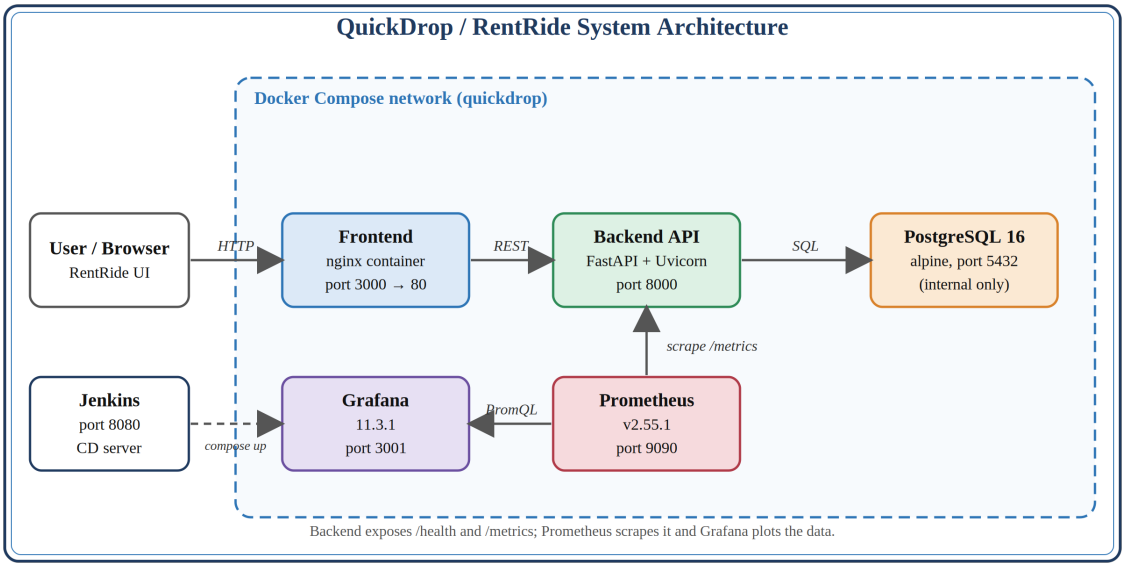



*Figure 3: System architecture of the Dockerised stack*

### 6.1 API Endpoints

The back end publishes interactive Swagger documentation at /docs. The main resource groups are listed below.

| Group | Endpoints | Purpose |
|---|---|---|
| categories | GET, POST /categories; GET, PUT, DELETE /categories/{id} | Manage product categories |
| products | GET, POST /products; GET, PUT, DELETE /products/{id} | Manage vehicles / products |
| cart | /cart, /cart/items | Add and view cart items |

| Group | Endpoints | Purpose |
|---|---|---|
| orders | /orders, /orders/checkout, /orders/{id}/status | Place orders, track status |
| system | /health, /metrics, /docs, /openapi.json | Health check, monitoring, documentation |

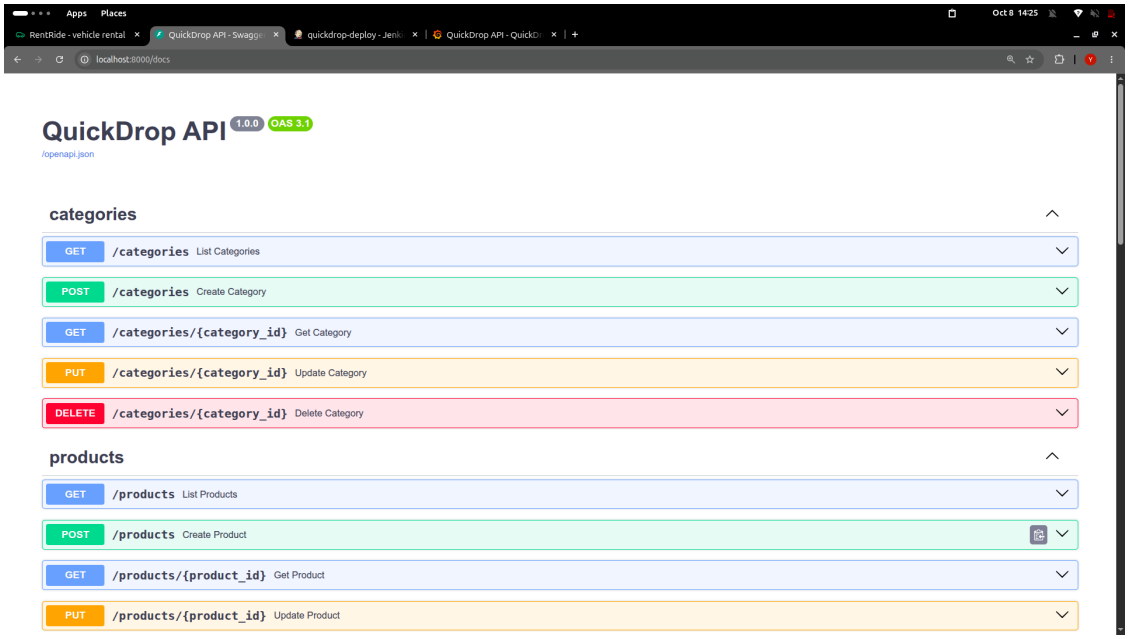



*Figure 4: Swagger UI of the QuickDrop API (localhost:8000/docs)*

## 7. Containerisation with Docker

Each component has its own container, which makes the environment identical on a laptop, on Jenkins and on AWS. The output of docker ps below shows the running services, with the back end and database reporting a healthy status.

| Container | Image | Port mapping |
|---|---|---|
| quickdrop-backend-1 | quickdrop-backend:local | 8000 → 8000 |
| quickdrop-frontend-1 | quickdrop-frontend:local | 3000 → 80 |
| quickdrop-grafana-1 | grafana/grafana:11.3.1 | 3001 → 3000 |
| quickdrop-prometheus-1 | prom/prometheus:v2.55.1 | 9090 → 9090 |
| quickdrop-db-1 | postgres:16-alpine | 5432 (internal) |
| quickdrop-jenkins-1 | quickdrop-jenkins:local | 8080 → 8080 |

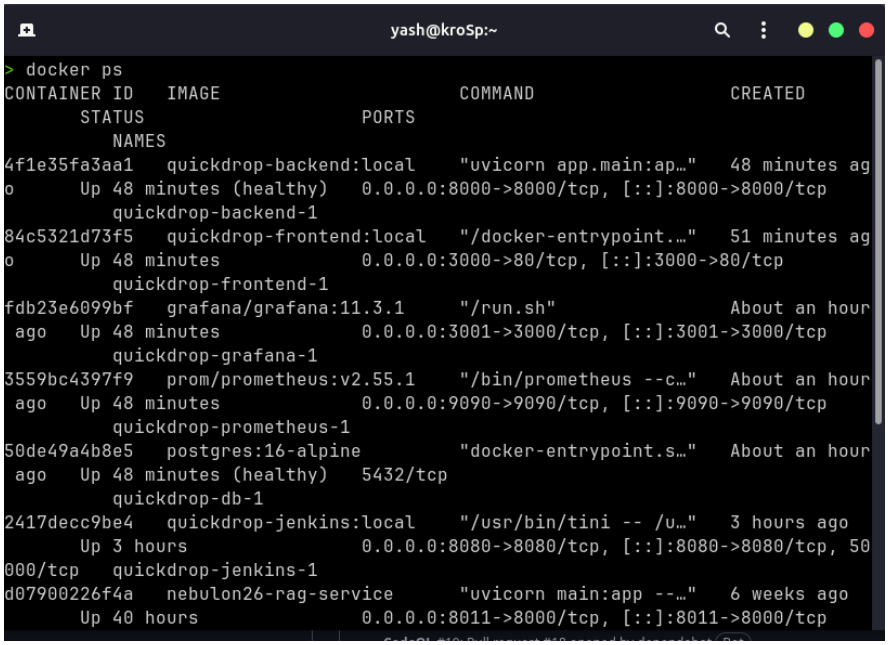



*Figure 5: docker ps output showing all running containers*

## 8. Continuous Integration with GitHub Actions

Workflows live in .github/workflows and start automatically on pushes and pull requests. They run in parallel, so a developer gets feedback within about five minutes. Dependabot is also enabled and opens pull requests when a dependency or action version is out of date; these pull requests pass through the same checks.

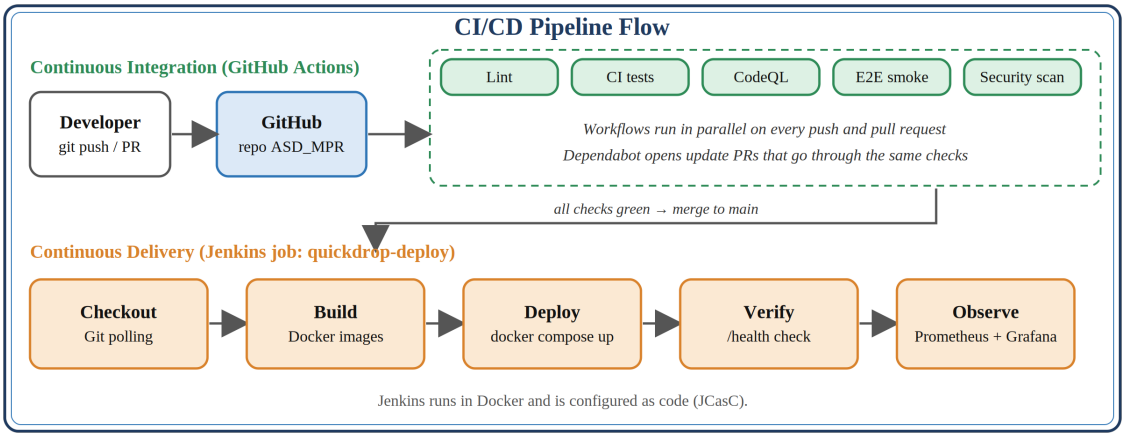



*Figure 6: CI/CD pipeline flow from commit to deployment*

| Workflow | What it does |
|---|---|
| CI | Installs dependencies and runs the automated tests |
| Lint | Checks code style and static quality |
| CodeQL | Static security analysis of the source code |
| E2E Smoke Test | Starts the stack and runs a booking / order smoke test end to end |
| Security Scan | Scans dependencies and images for known vulnerabilities |
| Dependabot Updates | Opens automatic version-bump pull requests |
| Release, Release Notes | Create releases and generate release notes |

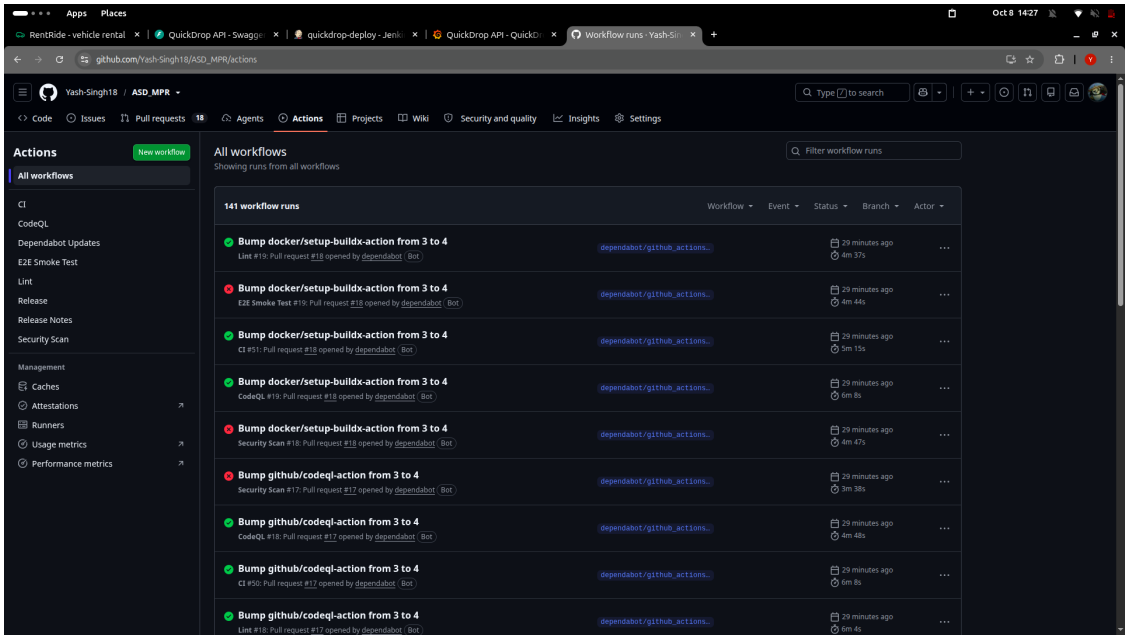



*Figure 7: GitHub Actions page showing 141 workflow runs*

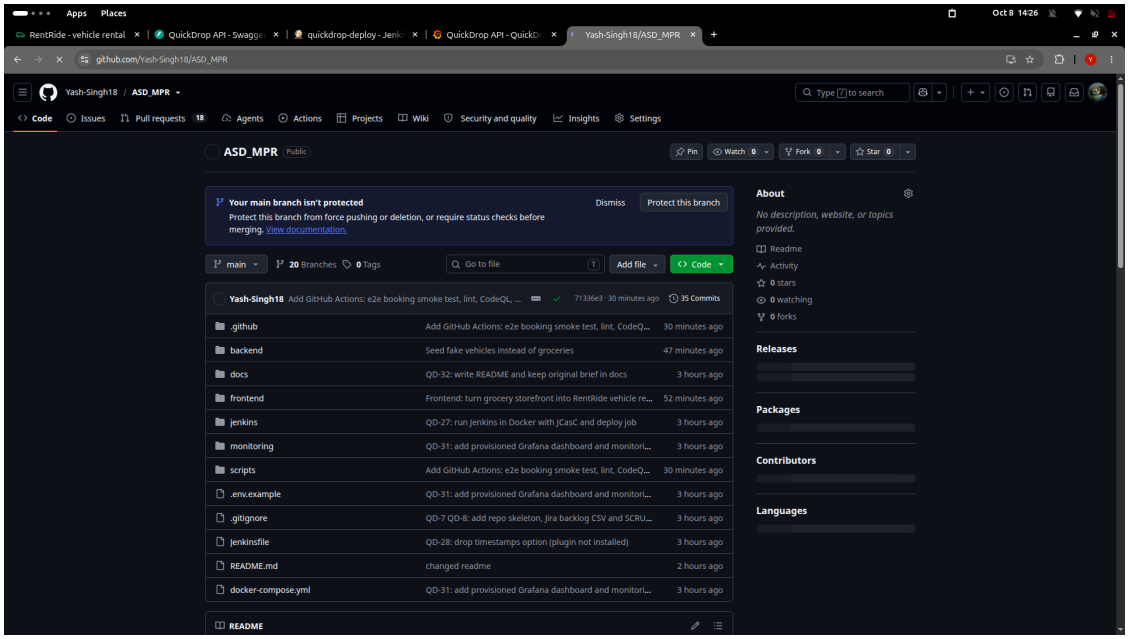



*Figure 8: Repository ASD_MPR with .github, backend, frontend, jenkins and monitoring folders*

**Observation:** In the runs captured, Lint, CI and CodeQL passed on the Dependabot pull requests, while the E2E Smoke Test (#19) and Security Scan (#17, #18) failed. These failures are exactly what CI is for: they flag changes that need a human review before merging. GitHub also reports that the main branch is not protected, so enabling branch protection with required status checks is the next improvement.

## 9. Continuous Delivery with Jenkins

Jenkins itself runs as a Docker container and is configured through Jenkins Configuration as Code (JCasC), so the whole server can be recreated from files in the jenkins folder. The pipeline is defined in the Jenkinsfile in the repository root and is used by the job quickdrop-deploy. It polls the Git repository, builds the Docker images, redeploys the stack with Docker

Compose and checks that the application is healthy.

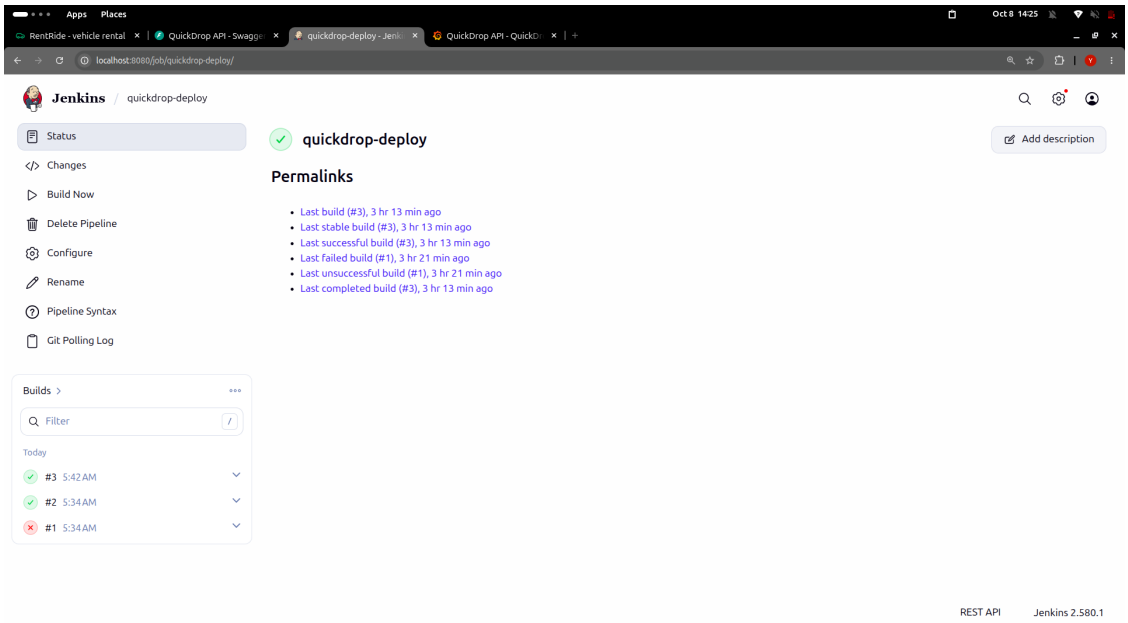



*Figure 9: Jenkins job quickdrop-deploy with its build history*

Build history: build #1 failed, and builds #2 and #3 succeeded. The first failure was caused by a pipeline option that needed a plugin which was not installed on the server (ticket QD-28). The option was removed from the Jenkinsfile and the next builds were green. This is a good example of fast feedback and fixing the pipeline as code.

## 10. Monitoring with Prometheus and Grafana

The FastAPI back end exposes request counters and latency histograms at /metrics. Prometheus scrapes them every few seconds and Grafana displays them on the QuickDrop API dashboard, which refreshes every 5 seconds.

| Dashboard panel | Meaning |
|---|---|
| Total requests | Number of HTTP requests served (612 in the captured window) |
| Server errors (5xx) | Failed requests caused by the server (0, so no server errors) |
| Backend up | Prometheus target status (1 = the backend is reachable) |
| Requests in progress | Requests being handled at that moment (0) |
| Request rate by endpoint | Requests per second for each API route |
| p95 latency by endpoint | 95% of requests complete faster than this time (about 100 ms) |
| Errors per second (4xx / 5xx) | Client and server error rate over time |

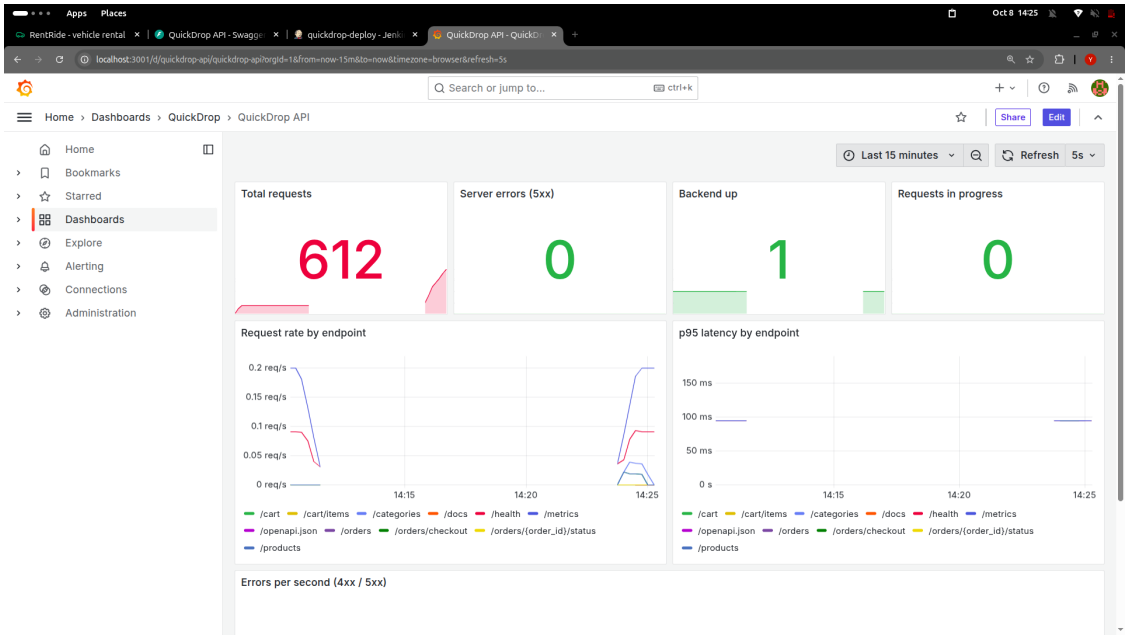



*Figure 10: Grafana dashboard "QuickDrop API" (localhost:3001)*

## 11. AWS Deployment

Because the whole application is already containerised, deploying to AWS needs no code change. The AWS side is prepared and can be executed whenever required. The planned steps are:

- Launch an EC2 instance (Ubuntu) and install Docker and Docker Compose.
- Create a Security Group that opens only the required ports (80, 3000, 8000 and 3001) and SSH from a trusted IP.
- Create an Amazon ECR repository; Jenkins builds the images and pushes them to ECR.
- On EC2, pull the images and start the stack with docker compose up -d.
- Add the EC2 public address to Prometheus and Grafana, and send logs to CloudWatch for alarms.

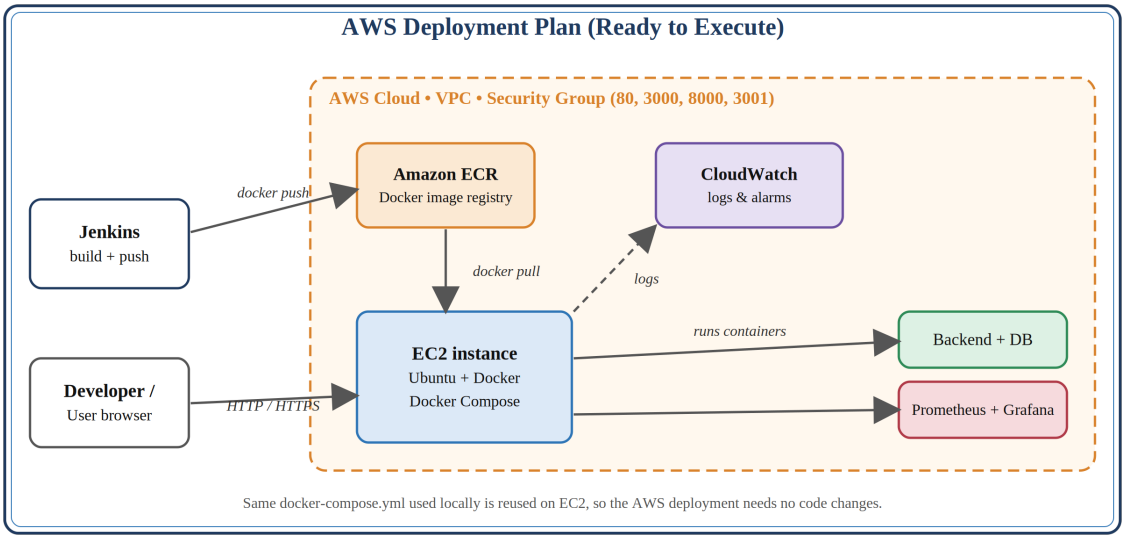



*Figure 11: Planned AWS deployment architecture*

**Note:** The AWS environment has been prepared but the final production deployment was not run for this report; the same docker-compose.yml that runs locally is used on EC2.

## 12. Results

| Area | Result |
|---|---|
| Planning | Backlog in Jira; commits linked to QD tickets |
| Source control | 35 commits, 20 branches, public GitHub repository |
| CI | 141 workflow runs across 8 workflows; most checks green |
| CD | Jenkins job quickdrop-deploy: 2 of 3 builds successful, latest build #3 successful |
| Containers | 6 containers running; back end and database healthy |
| Monitoring | 612 requests served, 0 server errors, backend up |
| Cloud | AWS target defined and ready |

## 13. Challenges and Learnings

- **Jenkins plugin missing:** the first pipeline run failed because of an unavailable plugin option; removing it fixed the build.
- **Failing automated checks:** Dependabot pull requests triggered smoke-test and security-scan failures, showing why changes should be reviewed before merging.
- **Unprotected main branch:** required status checks should be enforced so that failing changes cannot be merged.
- **Pipeline as code:** keeping Jenkins, Grafana and workflows as files in Git makes the setup repeatable and reviewable.

## 14. Conclusion

The mini project shows a complete Agile and DevOps cycle. Work is planned in Jira, every change is verified by GitHub Actions, delivered by Jenkins, run in Docker containers and observed through Prometheus and Grafana, with AWS ready as the hosting target. The result is a faster, more reliable and more transparent release process than manual deployment.

### Future Scope

- Enable branch protection and required checks on the main branch.
- Run the production deployment on AWS and automate it from Jenkins.
- Add Grafana alert rules for 5xx errors and high latency.
- Add automated rollback in the Jenkins pipeline.

## References

- GitHub Actions documentation, docs.github.com/actions
- Jenkins documentation, jenkins.io/doc
- Docker documentation, docs.docker.com
- Prometheus documentation, prometheus.io/docs
- Grafana documentation, grafana.com/docs

- Atlassian Jira Scrum guide, atlassian.com/agile/scrum
- AWS EC2 and ECR documentation, docs.aws.amazon.com